Repaso demo — lo que tienes que saber defendiendo

AquaSmart Mobile (Kotlin) + Arduino UNO R4 WiFi · TI3042 Sumativa U2 · Fri 06-11 16:15 · Repo: github.com/JJ-Molinet/aquasmart-mobile · Demo: 192.168.4.1 (AquaSmartAP / aquasmart123) · Login: juanjo / aquasmart123

1 · Mapa mental de la app (3 pantallas que muestro)

Pantalla	Qué hace	Puntos clave que defiendo
Login (LoginActivity)	Registrar + ingresar con usuario/contraseña	SQLite local (tabla `users`), contraseñas con **PBKDF2 + sal aleatoria**, bloqueo tras 3 intentos (anti fuerza bruta), sesión en **EncryptedSharedPreferences** (AES-256 con Android Keystore)
Menú (MenuActivity)	Elige rol del teléfono	"Modo Dispositivo IoT" (simulador BT), "Modo Monitor" (cliente BT + SQLite), "Monitor por WiFi" (HTTP → Arduino R4)
Monitor por WiFi (WifiArduinoActivity)	La demo principal: consulta estado y prende/apaga los 2 LEDs	HTTP GET a `http://192.168.4.1/status` y `/cmd?value=LED_ON`, parseo JSON en `org.json.JSONObject`, coroutines en `Dispatchers.IO` (la UI no se congela)

2 · Conceptos Kotlin/Android por si pregunta "cómo está hecho"

Concepto	Dónde va en mi código
Activity + ciclo de vida	`onCreate()` prepara pantalla (setContentView), `onDestroy()` cierra sockets y cancela coroutines
SQLite	`DbHelper` extiende `SQLiteOpenHelper`: `onCreate()` crea `users` y `lecturas`; inserts con `ContentValues`; consultas con `rawQuery`; datos de sensores persistidos para el historial
Cifrado de credenciales (PBKDF2)	`Security.hashPassword()`: `PBEKeySpec(password, salt, 120_000, 256)` + `SecretKeyFactory "PBKDF2WithHmacSHA256"`. Sal de 16 bytes con `SecureRandom` por usuario
Sesión cifrada	`Security.securePrefs()`: `EncryptedSharedPreferences` con `MasterKey AES256_GCM` (clave vive en Android Keystore, no exportable). La sesión expira a los 30 min
Control de acceso	Sin login válido no se entra al menú; bloqueo temporizado tras 3 intentos (`MAX_INTENTOS = 3`, 30 s)
hilos y corrutinas	Red/SQLite en `Dispatchers.IO`; actualización de UI con `runOnUiThread`/Handler. Regla de Android: nada de red en el hilo principal
Permisos Bluetooth (Android 12+)	`BLUETOOTH_CONNECT` es runtime en API 31+ (`Build.VERSION_CODES.S`): se pide con `ActivityCompat.requestPermissions`; el manifest declara versiones compatables por `maxSdkVersion`

3 · Conceptos Arduino (sketch R4 WiFi)

Función / línea	Qué hace / qué explicar
`pinMode` / `digitalWrite`	D12, D9, D8 como salidas; el LED responde con HIGH/LOW (D13 = LED "L" integrado también se apaga con el rojo)
`analogRead(A0)`	higrómetro 0-1023 → mapeado a % con calibración aire/agua (620/290)
`WiFi.config` + `WiFi.beginAP`	la placa crea el Access Point `AquaSmartAP` (WPA2) y se le fija la IP fija `192.168.4.1`
`WiFiServer server(80)`	servidor HTTP embebido; `server.available()` devuelve el cliente que conecta
`/cmd?value=LED_ON`	parseo del query: `indexOf("/cmd?value=")` → extraer valor → cambiar flags → `aplicarEstado()`
JSON de respuesta	`/status` y `/cmd` responden siempre JSON `{LED, LED2, bomba, T, HS, U, sensor}` — así la app siempre sabe el estado real de la placa (confirmación de comando)
220 Ω ¿por qué?	limita la corriente del LED (`R=(5V-2V)/15mA ≈ 200Ω`; 220 es el valor comercial más cercano). Sin ella el LED quemaría
REST / HTTP	cada comando es un GET cuyo "query string" lleva el comando; convención tipo REST (estado como recursos)

4 · El cuento que puedo narrar: cómo viaja un comando

> Botón 💡 → `comandar("LED_ON")` → coroutine en **IO** → `HttpURLConnection` → **GET http://192.168.4.1/cmd?value=LED_ON** → la **R4** parsea la línea de la petición → `ledRojo = true` → `digitalWrite(D12, HIGH)` → **el LED rojo**

> **del protoboard prende** → la placa responde JSON `{"LED":true,…}` → la app imprime "✅ Estado recibido" con el estado del LED2/bomba/T/HS.

Segundo cuento (Bluetooth): botón → `enviarComando("LED_ON")` → PrintWriter sobre **BluetoothSocket RFCOMM seguro** (UUID SPP) → el otro teléfono o el HC-05 lo recibe → responde `OK LED_ON`. El Monitor guarda cada lectura en SQLite y el historial las muestra.

5 · Seguridad ISO 27400 (si pregunta "y la seguridad?")

Control	Código
Cifrado de credenciales	contraseñas nunca en texto claro: PBKDF2-HMAC-SHA-256 (120k iter) con sal única
Datos de sesión en reposo	EncryptedSharedPreferences con clave AES-256-GCM protegida en Android Keystore
Control de acceso	sin login no hay comandos; bloqueo tras 3 intentos; sesión con expiración
Canal cifrado	BT: RFCOMM *secure* (`createRfcommSocketToServiceRecord`); WiFi: red del AP con passphrase WPA2 + `cmd/msg` validada por formato y lista blanca de comandos
Superficie de ataque	`allowBackup=false` , sin logs de credenciales, comandos validados (lista blanca); UART serial solo debug local

6 · Banco de preguntas probables del profe (con respuesta corta)

Sobre la app

¿Por qué SQLite y no Firebase/MySQL?

R: "SQLite es la opción local que el profe recomendó (tan 'súper fácil de integrar'); funciona sin internet (demo 100% inalámbrica local), cero latencia y cumple lo de la unidad. Firebase/MySql quedaría como evolución con respaldo en la nube."

¿Dónde guardas la contraseña?

En la tabla `users` de SQLite, pero NUNCA la cadena clara: guardo un hash PBKDF2 con sal de 16 bytes propia de cada usuario + iteraciones (120k) que hacen cara la fuerza bruta. La validación compara la derivación nueva con la guardada.

¿Qué es EncryptedSharedPreferences?

API de AndroidX que guarda preferencias cifradas: la clave AES-256-GCM vive en el **Android Keystore** (chip de seguridad, no exportable) y los "pares clave-valor" de la sesión van en disco cifrados.

¿Qué pasa si la app se cierra a mitad de sesión?

La sesión vive en el archivo cifrado, así que la próxima vez que abro la app entro sin re-login (token con timestamp; a los 30 min vence y pide login de nuevo).

¿Por qué coroutines/hilos?

Android bloquea la UI Thread: cualquier llamada a red o disco ahí genera un ANR. Con `Dispatchers.IO` las llamadas HTTP/SQLite van en un hilo worker y la UI actualiza con `runOnUiThread` .

¿Por qué pediste permiso Bluetooth en runtime?

Desde Android 12 (API 31) `BLUETOOTH_CONNECT` es permiso "dangerous": se declara en manifest y se pide en runtime. Antes (API ≤30) se usaba la pareja BLUETOOTH/BLUETOOTH_ADMIN con `maxSdkVersion` — mi manifest maneja las dos eras.

¿Qué es un Activity y por qué hay 4?

Cada pantalla es una Activity (Login, Menú, Dispositivo, Monitor, WiFi-Monitor). Cada una pide lo suyo en el manifest y ejecuta su `onCreate` . El que organiza es el Intent con `startActivity()` .

Sobre Arduino / la placa

¿Cómo le hago para que se conecte el celular a la App con la placa?

La R4 WiFi no hace Bluetooth "SPP" (solo BLE): por eso el canal que usamos es **WiFi**: el sketch crea un Access Point `AquaSmartAP` con la clave `aquasmart123` , el celular se conecta a esa red y la app habla con el servidor HTTP de la placa en `192.168.4.1` .

¿192.168.4.1 de dónde sale?

Con `WiFi.config(IPAddress(192,168,4,1))` fijé esa IP al AP: es la dirección típica de los routers caseros para su panel de configuración — el teléfono lo accede como si fuera un router. Se podría cambiar.

¿Cómo reacciona la placa al comando?

El servidor recibe la petición, parsea `/cmd?value=LED_ON`, pone el flag `ledRojo = true`, llama `aplicarEstado()` que hace `digitalWrite(D12, HIGH)` y devuelve el JSON con el estado real — o sea, hay "confirmación de comando" (no asume que funcionó: la placa reporta).

¿Porqué el LED "L" de la placa se prende igual?

En `aplicarEstado()` el D13 (LED_BUILTIN) sigue al LED rojo: es intencional — da feedback visual también sin protoboard (D12 = prototipo, D13 = placa).

¿Qué es el "/status"?

La app siempre puede pedir el estado vigente de la placa; así el app no "asume" el estado sino que consulta. Se devuelve como JSON (rep: LED, LED2, bomba, T, HS, sensor).

Sobre el circuito

¿De dónde sale 220 Ω y por qué?

Es el valor comercial más cercano a (5V − ~2V)÷15mA ≈ 200Ω que limita la corriente al LED. El resistor va en la pata CORTA (cátodo, −) hacia el riel azul (GND comprometido). Sin él el LED se quema.

¿Por qué dos resistencias si hay un solo GND?

Cada LED tiene SU corriente — la resistencia individual permite dos niveles de tensión y control independiente (LED rojo y LED amarillo). Comparten el GND igualmente.

¿Por qué los LEDs están al otro lado del canal central en la protoboard?

Para quedar en nodos DISTINTOS: en una protoboard cada grupo de 5 agujeros (a-e / f-j) es un nodo; si las dos patas caen en el mismo grupo, el LED queda cortocircuitado y no enciende nunca. Al cruzar el canal, separo los dos nodos.

¿Y si quisiera WiFi y no Bluetooth para la demo? ¿por qué es mejor WiFi?

Ambos cumplen: WiFi tiene más hilos directamente con un rango mayor y no depende de hardware adicional (el AP lo crea la propia placa). Bluetooth en la demo se puede usar con el HC-05 o con otro teléfono simulando, y muestra una tecnología distinta. La guía permite "Bluetooth o WiFi".

Sobre seguridad de la app

¿Es seguro el canal WiFi? ¿quinés leen el tráfico?

Es un AP WPA2 con passphrase propio; la superficie de ataque es mínima (red ad-hoc de un solo punto). La mejora natural es WPA3 + HTTPS con certificado propio (la demo es un prototipo educativo y se lo piden como justificación).

¿Qué pasa si alguien adivina mis credenciales de demo?

Las credenciales demo son públicas para la clase (usuario juanjo); en un producto final las saltaría con OAuth o regreset de contraseña. Igual el registro crea cuentas nuevas con hash PBKDF2.

7 · Checklist del día (lo que muestro en orden)

```
1.-Login juanjo/aquasmart123 (PBKDF2 + Keystore)
2.-Menú → "Monitor por WiFi" → IP 192.168.4.1 → Conectar → ✅ estado JSON
3.-💡 Prender LED (rojo D12)` + L de la placa / 💡💡 LED 2 (amarillo D9)
4.-Log del comando y respuesta Arduino (bidireccional)
5.-Si pregunta seg.: tabla de 5 mapeos ISO 27400 (25 ptos)
6.-Repo GitHub + Entrega_TI3042_S2.txt listo pa AAI
```